Information Sciences 751 (2026) 123554

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Adaptive meta-path-based neural network architecture search for heterogeneous graphs

Xinsheng Li [a], Pietro Lió [b], Lintao Yang [a], Zhigang Ye [a], Chengbin Peng [a,*]

[a] Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo 315211, China
[b] Department of Computer Science and Technology, University of Cambridge, Cambridge, CB3 0FD, UK

HIGHLIGHTS

- A dependency-aware HGNAS framework is proposed for adaptive meta-path search.
- Meta-path combinations are selected to reduce redundancy and dependency.
- Adaptive aggregation is optimized jointly within a unified search process.
- AMP-HGNAS outperforms state-of-the-art HGNNs on benchmark datasets.

ARTICLE INFO

Keywords:
Heterogeneous graph neural networks
Heterogeneous graph representation learning
Neural architecture search

ABSTRACT

Many heterogeneous graph neural networks aim to capture the complex relationships between nodes and edges by utilizing predefined meta-paths, which help enhance the learning of heterogeneous graph representations. In this context, adaptive meta-path aggregation is crucial for heterogeneous graph neural networks, as it enables models to emphasize task-relevant semantic relations while reducing redundancy from irrelevant or correlated meta-paths. However, relying on manually selected meta-paths limits the flexibility and adaptability of these models. To address this limitation, we propose Adaptive Meta-Path for Heterogeneous Graph Neural Architecture Search (AMP-HGNAS), a framework that adaptively discovers and aggregates task-specific meta-paths in a unified architecture search process. Specifically, we introduce (1) a mechanism for identifying the most informative combinations of meta-paths to enhance representation learning while reducing redundancy, and (2) an adaptive meta-path aggregation based on meta-path dependencies that dynamically optimizes the use of selected meta-paths to improve the overall performance of the model. Extensive experiments on multiple benchmark datasets demonstrate that AMP-HGNAS achieves superior performance in multi-class classification tasks compared to existing state-of-the-art HGNN methods.

1. Introduction

Heterogeneous graphs [1] play a key role in real-world applications where they are used to accurately model diverse entities and their interactions in complex systems. To handle the complexity of these heterogeneous structures, heterogeneous graph neural networks (HGNNs) [2] have been introduced, which leverage rich meta-paths to model intricate relationships between different types of nodes and edges. For example, in the DBLP academic network [3], various node types exist, including Authors (A), Papers (P), Terms

* Corresponding author.
Email addresses: 2311100292@nbu.edu.cn (X. Li), Pietro.Lio@cl.cam.ac.uk (P. Lió), 2211100291@nbu.edu.cn (L. Yang), 2311100316@nbu.edu.cn (Z. Ye), pengchengbin@nbu.edu.cn (C. Peng).

https://doi.org/10.1016/j.ins.2026.123554
Received 4 November 2025; Received in revised form 22 April 2026; Accepted 23 April 2026
Available online 24 April 2026
0020-0255/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(T), and Venues (V), along with diverse edge types such as collaborations between authors and papers, citation relationships between papers, authors' participation in conferences, and associations between papers and research topics. These different types of nodes and edges can be combined in specific sequences to form meta-paths with clear semantics. For example, the meta-path A–P–T–P–A (Author–Paper–Term–Paper–Author) is a 4-hop schema path that captures indirect relationships between authors through co-authored papers and associated terms, helping to reveal collaboration networks among researchers. Similarly, the meta-path A–P–A represents the relationship between two authors who have co-authored an academic paper. By constructing and analyzing these meta-paths, deeper insights into the complex interactions can be gained. However, the usefulness of meta-paths is highly context-dependent: the optimal meta-path combinations and fusion behaviors vary across datasets and downstream tasks. Therefore, adaptive meta-path aggregation is critical for HGNNs to leverage heterogeneous semantics effectively while avoiding redundant or noisy meta-path messages.

Existing studies mainly rely on manually designed architectures to encode structural and semantic information in heterogeneous graphs, as exemplified by HAN [4], MAGNN [5], and subsequent meta-path-based variants such as MECCH [6]. Although these methods have achieved promising results, their dependence on handcrafted architectures restricts their adaptability across diverse graph structures and various tasks. Moreover, as the complexity of models increases, straightforward feature combinations may introduce noise, thereby affecting the overall performance of the models.

The indiscriminate use of all meta-paths can incur heavy computation and introduce noise and dependencies. Several lines of work try to reduce the burden. SeHGNN [7] uses a simplified single-layer formulation with meta-paths up to a fixed maximum hop, but the hop length is typically kept small on large graphs, and the meta-path set remains fixed rather than task-adaptive. HAGNN [8] adaptively fuses multiple manually specified meta-path graphs, yet it still relies on pre-defined meta-path candidates and does not explicitly control redundancy among correlated meta-paths. LMSPS [9] progressively samples and selects meta-paths; however, the sampling-based procedure can be stochastic, and it does not explicitly model inter-meta-path dependencies, which may result in the retention of highly correlated paths. Beyond path selection, NAS-based approaches aim to automate architectural choices. HGNAS [10] designs a new search space and uses a policy network to sample candidate architectures, and HGNAS++ [11] further employs an RNN [12] controller within a generative adversarial framework plus a pairwise ranker to cut evaluation costs. Broader automatic strategies [13] reduce expert dependence by automating meta-path/aggregation design. Even CCMG [14], which co-evolves meta-multigraphs and architectures, and SchemaWalk [15], which uses RL-based schema navigation for meta-paths, do not explicitly model inter-meta-path dependency, and their aggregation design is not jointly optimized with dependency-aware selection. However, most existing HGNAS methods still construct or explore meta-path candidates without explicitly modeling inter-meta-path dependency or jointly optimizing dependency-aware path selection with aggregation design. This limitation becomes more pronounced on large-scale benchmarks, where scalability is also a major concern. In contrast, our method introduces dependency-aware meta-path selection and unified aggregation search within a single framework.

To tackle these issues, we propose a meta-path-based neural architecture search framework for heterogeneous graphs, which decouples the problem into dependency-aware meta-path selection for eliminating redundant meta-paths and improving computational efficiency, together with a module-level differentiable search mechanism that learns task-specific feature fusion strategies, enhancing adaptability across datasets and tasks. In this work, we define the dependency between two meta-paths as their structural redundancy, measured by the extent of overlap between their target-centered reachable subgraphs. This indicates that the two meta-path channels convey strongly correlated messages, which introduce information duplication and yield diminishing returns under joint aggregation.

To the best of our knowledge, this is the first approach in heterogeneous graph neural networks to integrate meta-path dependency and aggregation strategies into the search space. The main contributions of our method can be summarized as follows:

- We propose a meta-path combination selection strategy that models inter-meta-path dependencies via a meta-path dependency graph and adaptively selects high-utility meta-path combinations for task-specific node representation learning.
- We propose an adaptive meta-path aggregation strategy that leverages a module-level DARTS algorithm to optimize meta-path aggregation operators for specific downstream tasks.
- Experiments on four public heterogeneous graph datasets, including OGBN-MAG, demonstrate the effectiveness and scalability of our proposed method.

2. Related work

2.1. Graph neural architecture search

Graph Neural Architecture Search (GNAS) [16] is an intricate and automated methodology designed to uncover and optimize neural network architectures that are exquisitely matched to the minute details of specific datasets, effectively addressing the intricacies and subjectivity inherent in manual architectural design. In homogeneous graph neural networks, NAS [17] has been widely used to adaptively discover network architectures suitable for predefined tasks. GraphNAS [18] has proposed two types of search spaces, namely, meta-operator space and GNN cell space. The meta-operator space consists of message encoding operators and message aggregation operators. AM-GNAS [19] combines multi-scale message passing with adaptive weight adjustment to effectively explore and utilize both low-pass and high-pass information in graph structures. In addition, it is equally important to apply NAS to heterogeneous graph neural networks, given that the diversity of node and edge types in heterogeneous graphs raises the complexity of manually designing neural network architectures. Heterogeneous graph neural architecture search [20] proposes a multi-stage search algorithm specifically for the architecture design and hyperparameter adjustment of heterogeneous graph neural networks, achieving fully automated architecture design and hyperparameter adjustment. HGNAS [10] constructs a new search space and uses a policy

network as a controller to sample and identify the best architecture. HGNAS++ [11] further improves this process by training an RNN controller within a generative adversarial learning framework and integrating a pairwise ranker to filter candidate architectures, effectively reducing the number of candidate architecture evaluations and search time. LMSPS [9] focuses on meta-path search through a progressive sampling algorithm, reducing the number of meta-paths from exponential to a subset effective for specific datasets and tasks. CCMG [14] further explores jointly evolving meta-structure construction and aggregation architectures. However, these methods lack personalized architecture search based on meta-paths and thus struggle to accommodate varying meta-path combinations and datasets.

2.2. Heterogeneous graph neural networks

Heterogeneous Graph Neural Networks aim to extract and encode rich semantic information from heterogeneous graphs to support complex downstream tasks. Meta-path-based HGNNs, such as HAN [4], MAGNN [5], HMSG [21], MEGNN [22], and HetGNN [23], enable higher-order semantic aggregation, yet their deployment on large-scale datasets is often constrained by high computational costs. Meanwhile, models such as SR-HGN [24], HGT [25], and GTN [26] implicitly capture meta-path-like semantics via attention, but the discovered patterns are rarely used explicitly in final prediction, and are even less often transferred to other backbones for verification. For example, HGT [25] starts from one-hop directed edges and aggregates higher-order neighbors through multi-layer heterogeneous attention, which can suffer from limited effective hop depth, coarse semantic granularity, and the lack of explicit search, whereas HAN [4], HMSG [21], and MAGNN [22] mainly validate meta-path importance with auxiliary analyses that are not tightly coupled to task loss, leaving limited quantitative evidence on end-task gains or statistical stability. However, existing HGNN methods often ignore inter-meta-path dependencies and lack a unified framework for jointly optimizing meta-path selection and aggregation. Our method addresses this by explicitly modeling meta-path dependencies and unifying dependency-aware selection with adaptive aggregation search.

3. Method

In this work, we propose a novel Adaptive Meta-Path-based Neural Architecture Search for Heterogeneous Graphs (AMP-HGNAS). This method comprehensively considers the dependencies between meta-paths during the meta-path search optimization process and devises an adaptive meta-path aggregation strategy to address the aforementioned issues. Unlike existing meta-path-based heterogeneous GNN architecture search approaches that treat meta-paths as independent candidates or explore them without modeling their mutual correlations, AMP-HGNAS explicitly constructs a meta-path dependency graph to perform redundancy-aware selection, reducing over-correlated path combinations during the search. Furthermore, rather than decoupling meta-path construction from aggregation design, our framework conducts module-level differentiable search to jointly learn task-adaptive meta-path aggregation and fusion. An overview of the framework is illustrated in Fig. 1.

3.1. Preliminaries

Heterogeneous Graph Neural Networks (HGNNs) [27] can be modeled as a directed graph $G = (\mathbf{V}, \mathbf{E})$, where each node $v \in \mathbf{V}$ is assigned a unique node type $\phi(v) \in \mathbf{A}$ and each edge $e \in \mathbf{E}$ is assigned a unique relationship type $\psi(e) \in \mathbf{R}$. Here, $\mathbf{A}$ denotes the set of node types, and $\mathbf{R}$ denotes the set of relationship types. The network is defined as a heterogeneous information network [28] when the number of node types $|\mathbf{A}| > 1$ or the number of relationship types $|\mathbf{R}| > 1$. Conversely, if either $|\mathbf{A}| = 1$ or $|\mathbf{R}| = 1$, indicating all nodes or edges belong to a single type, the network is considered homogeneous.

Meta-paths [29] are referred to as an ordered sequence of node types used to describe the various relationships between different types of nodes in a heterogeneous information network, which can be represented as $v_1 \xrightarrow{e_1} v_2 \xrightarrow{e_2} \cdots \xrightarrow{e_l} v_{l+1}$. Taking a literature information network as an example, v_1 can be an author, v_2 can be a paper, and v_3 can be a venue. Here, e_1 represents the relationship "write," indicating that an author has authored a paper, and e_2 represents the relationship "published in," indicating that the paper has been published in a certain venue. Such a directed path not only reflects the direct connection between the author and the work but also reveals the channels through which the work is disseminated.

Dependency definition. For each meta-path P_K, we define the meta-path-reachable subgraph rooted at the target node type as $G(P_K) = (N(P_K), E(P_K))$, where $N(P_K)$ denotes the set of nodes reachable via instances of P_K, and $E(P_K)$ denotes the set of edges traversed by these instances. In this work, we focus on structural dependency between meta-paths, which captures redundancy in their induced neighborhoods. Given two meta-paths P_i and P_j, their structural dependency is quantified by a similarity-based redundancy score computed from $N(P_i), N(P_j)$ and $E(P_i), E(P_j)$, so that a larger score indicates stronger structural dependency and a higher likelihood of duplicated messages under joint aggregation.

3.2. Stage I: selecting meta-path combinations

Stage I performs dependency-aware meta-path combination selection to obtain a compact yet diverse meta-path set. Unlike prior approaches that rely on manually specified meta-paths or unconstrained sampling, we explicitly control inter-meta-path structural dependency by selecting a set of non-adjacent meta-paths on the dependency graph, thereby avoiding multiple highly dependent meta-path channels and reducing redundant computation.

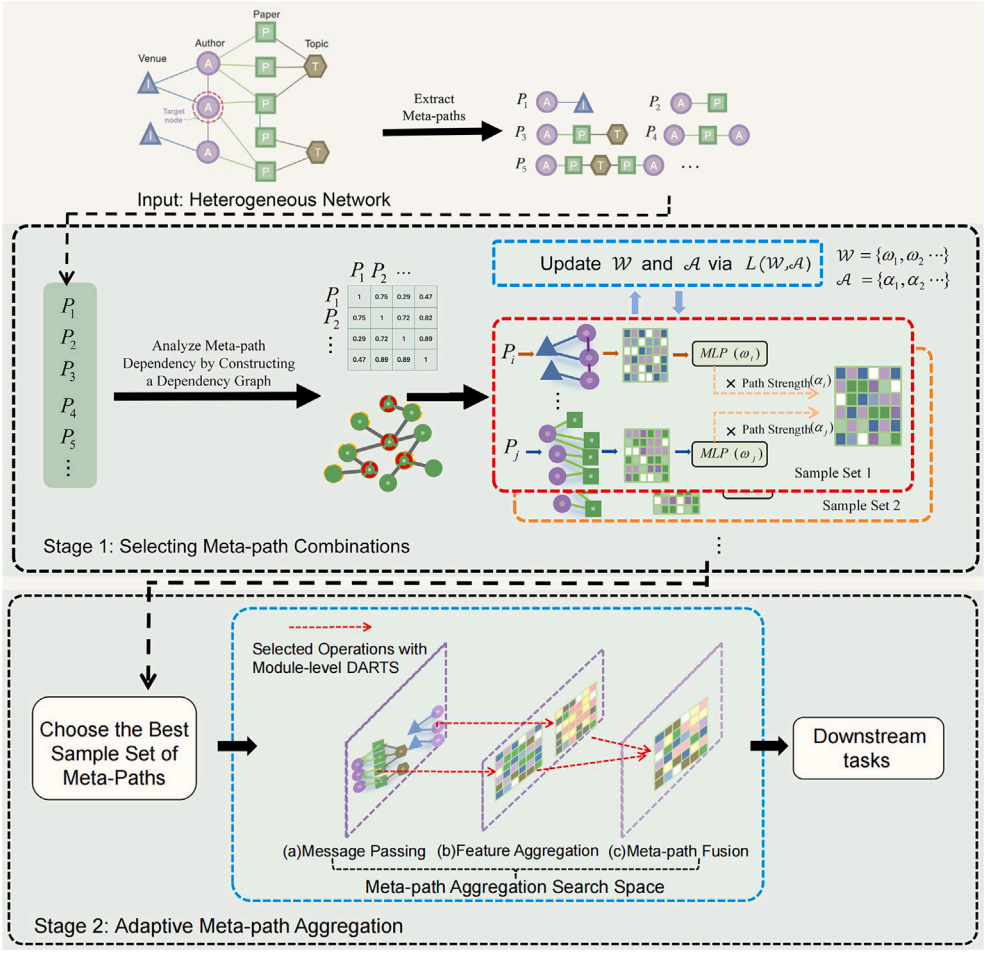


Fig. 1. The overall framework of AMP-HGNAS. For a heterogeneous network, the processing framework is composed of two key stages: **Stage I: Meta-path combination selection** focuses on dependency-aware redundancy control: we first construct a meta-path dependency graph by analyzing inter-meta-path dependencies, and then sample candidate meta-paths with diverse activity to reduce redundancy. Each candidate meta-path is associated with learnable parameters α and ω. **Stage II: Adaptive meta-path aggregation** focuses on task-adaptive aggregation and fusion: we employ a module-level differentiable architecture search to determine the aggregation search space and operations tailored to the downstream task.

The initial set of meta-paths is generated by considering all possible meta-paths associated with the target node; we use $\mathbf{P} = \{P_1, P_2, \ldots, P_K\}$ to denote the K different meta-paths associated with the target node. Specifically, each meta-path P_K can be defined as $P_K = \{v_1^k, e_1^k, v_2^k, e_2^k \ldots, e_{l-1}^k, v_l^k\}$, where v_1^k is the starting node and v_l^k is the ending node.

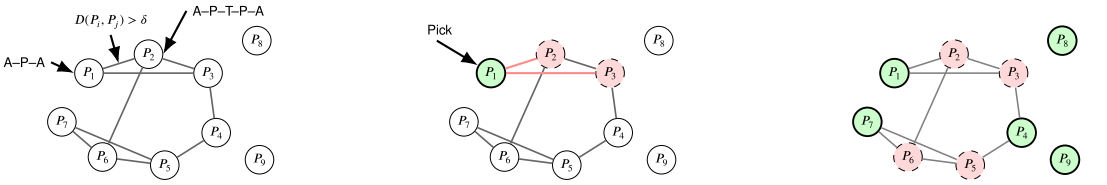
Constructing dependency matrix. For any two meta-paths $P_i, P_j \in \mathbf{P}$, their pairwise dependency $D(P_i, P_j)$ is computed as the product of two Jaccard similarities-one over the node sets and the other over the edge sets-covered by the meta-paths:

$$D(P_i, P_j) = \frac{|N(P_i) \cap N(P_j)|}{|N(P_i) \cup N(P_j)|} \cdot \frac{|E(P_i) \cap E(P_j)|}{|E(P_i) \cup E(P_j)|}, \tag{1}$$

where $N(P_i)$ and $N(P_j)$ represent the sets of nodes covered by meta-paths P_i and P_j respectively, while $E(P_i)$ and $E(P_j)$ represent the sets of edges covered by meta-paths P_i and P_j respectively. We can construct a dependency matrix D to quantify the dependency between different meta-paths. The dependency matrix D is defined as follows:

$$D = \begin{bmatrix} D(P_1, P_1) & D(P_1, P_2) & \cdots & D(P_1, P_K) \\ D(P_2, P_1) & D(P_2, P_2) & \cdots & D(P_2, P_K) \\ \vdots & \vdots & \ddots & \vdots \\ D(P_K, P_1) & D(P_K, P_2) & \cdots & D(P_K, P_K) \end{bmatrix}. \tag{2}$$

Constructing a meta-path dependency graph. Based on the calculated meta-path dependency matrix, we can construct a meta-path dependency graph where nodes represent meta-paths and edges represent high dependency between meta-paths. If the dependency degree $D(P_i, P_j)$ between meta-paths P_i and P_j is greater than the preset threshold δ, then an edge is added to the meta-path dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, indicating that there is a dependency relationship between them. This process can be expressed



(a) Meta-path dependency graph. (b) Redundancy graph (δ=0.7). (c) $S_1 = \{P_1, P_4, P_7, P_8, P_9\}, \cdots$.

Fig. 2. Meta-path dependency modeling and redundancy-aware sampling on DBLP (target node type: Author). Each node represents an author-centered candidate meta-path, e.g., P_1: A–P–A (co-author: two authors write the same paper), P_2: A–P–T–P–A (shared terms: two authors are linked via papers containing the same keywords), and P_3: A–P–V–P–A (shared venues: two authors publish in the same venue). An edge indicates strong dependency $D(P_i, P_j) > \delta$ (Eq. (3)), i.e., the two meta-paths induce highly overlapping neighborhoods. **(a) Meta-path dependency graph** with the edge criterion $D(P_i, P_j) > \delta$. **(b) Initial selection:** start from an unselected meta-path (green) and mark all its adjacent meta-paths as excluded (red); isolated nodes (P_8, P_9) are always kept since they have no neighbors. **(c) Selecting the least-correlated meta-path set:** an example non-adjacent set $S_1 = \{P_1, P_4, P_7, P_8, P_9\}$ obtained from one greedy run, where selected meta-paths are mutually non-adjacent (i.e., least correlated under the dependency graph).

with the following formula:

$$\mathcal{E} = \{(\mathcal{V}_{P_i}, \mathcal{V}_{P_j}) \mid D(P_i, P_j) > \delta, \forall \mathcal{V}_{P_i}, \mathcal{V}_{P_j} \in \mathcal{V}\}, \tag{3}$$

where $\mathcal{V}$ is the set of nodes in the meta-path dependency graph, representing the collection of all meta-paths, $\mathcal{V}_{P_i}$ is the node corresponding to the meta-path P_i in the dependency-based graph $\mathcal{G}$, and $\mathcal{E}$ is the set of edges in the meta-path dependency graph, representing the dependency relationships between meta-paths. An illustrative example on the DBLP benchmark is shown in Fig. 2.

Meta-path sampling and initialization. During the meta-path sampling process, the primary task is to address the issue of over-correlated meta-paths. To this end, we transform the meta-path sampling problem into the problem of finding an optimal set of non-adjacent nodes on the meta-path dependency graph we have obtained. We aim to obtain a set of non-over-correlated meta-path combinations S with an algorithm to reduce resource consumption in subsequent computational processes. This can be succinctly represented as:

$$S^* = \arg\max_{\mathcal{T} \subseteq \mathcal{V}} |\mathcal{T}| \quad \text{s.t.} \quad \forall \mathcal{V}_{P_i}, \mathcal{V}_{P_j} \in \mathcal{T},\ i \neq j\ :\ (\mathcal{V}_{P_i}, \mathcal{V}_{P_j}) \notin \mathcal{E}, \tag{4}$$

where S^* denotes the maximum independent set of the dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, which is the largest subset of meta-path nodes such that no two selected nodes are connected by an edge. In practice, we compute S using a greedy solver $\Phi(\cdot)$, namely $S = \Phi(\mathcal{G})$, which returns a maximal independent set.

Next, we define q_k as the meta-path strength of meta-path P_K:

$$q_K(\alpha_K) = \frac{\exp\left(\left(\alpha_K + \frac{|N(P_K)|+|E(P_K)|}{2}\right)/\tau\right)}{\sum_{j=1}^{K} \exp\left(\left(\alpha_j + \frac{|N(P_j)|+|E(P_j)|}{2}\right)/\tau\right)}, \tag{5}$$

where $\mathcal{A} = \{\alpha_1, \alpha_2, \ldots, \alpha_K\}$ denotes the architectural parameters related to meta-paths, preventing the model from favoring long meta-paths and ignoring semantically important shorter meta-paths, τ is the temperature parameter that controls the smoothness of the distribution, and K denotes the total number of meta-paths in the set S. The larger the node cover set size $|N(P_K)|$ and edge cover set size $|E(P_K)|$ of a meta-path P_K, the more heterogeneous graph information it contains, and thus the higher its meta-path strength.

Node representation with selected meta-paths. First, we define the feature matrix of the K-th meta-path P_K as $\hat{X}_K$, which is obtained through the neighbor aggregation process:

$$\hat{X}_k = \left(\prod_{i=0}^{l-1} \hat{A}_{v_i^k, v_{i+1}^k}\right) \mathcal{X}_{v_l^k}, \tag{6}$$

where $\mathcal{X}_{v_l^k}$ is the raw feature of nodes of type v_l^k, and $\hat{A}_{v_l^k, v_{i+1}^k}$ denotes the row-normalized adjacency matrix between node types v_i^k and v_{i+1}^k.

In the forward propagation process, each node aggregates information from its neighboring nodes based on the selected meta-paths and the strengths of these meta-paths q_k. This can be achieved by using a multi-layer perceptron (MLP) to update the representation of the nodes:

$$\hat{Z}_v = \mathcal{F}_{\omega_1}\left(\sum_{k \in S} q_k \cdot \mathcal{F}_{(\omega_2,k)}(\hat{X}_k)\right), \tag{7}$$

where $\hat{Z}_v$ corresponds to the updated representation of the target node v, and $\mathcal{W} = \{\omega_1, \omega_2\}$ represents the weight parameters in $\mathcal{F}$. For each selected meta-path feature $\hat{X}_k$, we first use $\mathcal{F}_{(\omega_2,k)}$ to project different feature matrices onto the same dimensional space. Subsequently, all processed features are aggregated by a summation operation. Finally, an external $\mathcal{F}_{\omega_1}$ is applied to further integrate these features to obtain the final node representation $\hat{Z}_v$.

Optimization for $\mathcal{A}$ and $\mathcal{W}$. We adopt a bilevel optimization scheme to jointly learn the network weights $\mathcal{W}$ and the architecture parameters $\mathcal{A}$. For node classification, given the logits $\hat{Z}_v^i$ for node i, the predicted probability is

$$\hat{y}_i = \frac{e^{\hat{Z}_v^i}}{\mathbf{1}^T e^{\hat{Z}_v^i}}. \tag{8}$$

Accordingly, the cross-entropy loss on a dataset split D is

$$L_D(\mathcal{W}, \mathcal{A}) = -\frac{1}{|D|} \sum_{i \in D} \sum_{c=1}^{C} y_{ic} \log(\hat{y}_{ic}), \tag{9}$$

where C is the number of classes and $\mathcal{D} \in \{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}\}$ denotes the corresponding data split. Following differentiable neural architecture search (DARTS), we update $\mathcal{W}$ on the training split and update $\mathcal{A}$ on a held-out validation split to promote generalization and to avoid overfitting the searched architecture to the training loss; the test split is only used for final evaluation. Specifically, we apply alternating updates (a standard approximation used in DARTS):

$$\mathcal{W}^{(t+1)} = \arg\min_{\mathcal{W}} L_{\text{train}}(\mathcal{W}, \mathcal{A}^{(t)}), \tag{10}$$

$$\mathcal{A}^{(t+1)} = \arg\min_{\mathcal{A}} L_{\text{val}}(\mathcal{W}^{(t+1)}, \mathcal{A}), \tag{11}$$

where $t \in \{1, 2, \ldots, T\}$.

The pseudo-code of the proposed method in the meta-path search phase is described in Algorithm 1.

3.3. Stage II: adaptive meta-path aggregation

Stage II searches for task-adaptive aggregation and fusion given the meta-path set selected in Stage I. In contrast to conventional HGNN pipelines that rely on hand-crafted and fixed aggregation/fusion operators once meta-paths are specified, AMP-HGNAS further learns how to aggregate and fuse messages in a data-driven manner. Concretely, we formulate heterogeneous aggregation as three semantics-aligned modules. Each module is equipped with a small set of candidate operators and optimized via a differentiable selection mechanism, enabling the model to automatically specialize its aggregation behavior to different datasets while keeping the search space compact and stable.

3.3.1. Module-level search space for meta-path aggregation

Unlike traditional whole-space search, we divide the architecture into three compact modules: message passing, aggregation, and fusion, and search each separately, thereby reducing parameters, facilitating convergence, and revealing meta-path importance.

(a) Message Passing Functions. They can pass information based on the importance of semantic relationships, and examples in $\mathcal{O}^{(1)} = \{o_1^{(1)}, o_2^{(1)}, o_3^{(1)}\}$ are as follows.

Direct Aggregation considers the starting point and endpoint of the path, ignoring the intermediate node information in the path:

$$\tilde{X}_k^{(1)} = o_1^{(1)}\left(\mathcal{X}_v \mid v \in P_k\right) = \left(\prod_{i=0}^{l-1} \hat{A}_{v_i^k, v_{i+1}^k}\right) \mathcal{X}_{v_l^k}, \tag{12}$$

where l represents the length of the corresponding meta-path.

Algorithm 1 Adaptive Meta-Path Sampling Algorithm.

Require: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, threshold δ, iterations T

Compute the dependency matrix D based on Eq. (1)
Construct the dependency graph $\mathcal{G}_r = (\mathcal{V}, \mathcal{E}_r)$ based on Eq. (3)
Initialize $\mathcal{S} = \emptyset$
for $P_i \in \mathcal{V}$ **do**
    if P_i is not selected **then**
        Add P_i to $\mathcal{S}$
        Remove P_i and its neighbors from $\mathcal{V}$
    end if
end for
Compute the meta-path weights based on Eq. (5) and update node representations $\hat{Z}$ based on Eq. (7)
for $t = 1$ to T **do**
    Fixed architecture parameter $\mathcal{A}^{(t)}$, update network weight $\mathcal{W}^{(t+1)}$ according to Eq. (10)
    Fixed network weight $\mathcal{W}^{(t+1)}$, update architecture parameter $\mathcal{A}^{(t+1)}$ according to Eq. (11)
end for

Mean Aggregation generates new aggregated features by element-wise averaging of all nodes' features along a meta-path:

$$\tilde{X}_k^{(1)} = o_2^{(1)}\left(\mathcal{X}_v \mid v \in P_k\right) = \frac{1}{l+1}\sum_{i=0}^{l}\left(\prod_{j=0}^{i-1}\hat{A}_{v_j^k, v_{j+1}^k}\right)\mathcal{X}_{v_i^k}. \tag{13}$$

MLP Transformation applies a multi-layer nonlinear mapping to enhance feature expressiveness and capture complex semantics:

$$\tilde{X}_k^{(1)} = o_3^{(1)}\left(\mathcal{X}_v \mid v \in P_k\right) = \sum_{i=0}^{l}\mathrm{MLP}\left(\left(\prod_{j=0}^{i-1}\hat{A}_{v_j^k, v_{j+1}^k}\right)\mathcal{X}_{v_i^k}\right). \tag{14}$$

(b) Feature Aggregation Functions. They consolidate node features within each meta-path instance. Some examples are defined as $\mathcal{O}^{(2)} = \{o_1^{(2)}, o_2^{(2)}, o_3^{(2)}\}$.

Mean averages all meta-path instances element-wise to produce a unified feature representation:

$$\tilde{X}_k^{(2)} = o_1^{(2)}\left(\left\{\tilde{X}_k^{(1)}\right\}_{k=1}^{\eta}\right) = \frac{1}{\eta}\sum_{k=1}^{\eta}\tilde{X}_k^{(1)}. \tag{15}$$

Max Pooling takes the element-wise maximum across all meta-path instances to highlight the most salient features:

$$\tilde{X}_k^{(2)} = o_2^{(2)}\left(\left\{\tilde{X}_k^{(1)}\right\}_{k=1}^{\eta}\right) = \max\left\{\tilde{X}_k^{(1)}\right\}_{k=1}^{\eta}. \tag{16}$$

Attention uses learnable weights $\theta_{vu}^{P_k}$ computed by standard LeakyReLU attention and softmax for weighted summation:

$$\tilde{X}_k^{(2)} = o_3^{(2)}\left(\left\{\tilde{X}_k^{(1)}\right\}_{k=1}^{\eta}\right) = \sigma\left(\sum_{u\in\mathcal{N}_v^{P_k}}\theta_{vu}^{P_k}\tilde{X}_k^{(1)}\right), \tag{17}$$

where η denotes the number of meta-path P_k instances, $\tilde{X}_k^{(1)}$ is the feature vector of the k-th instance, $\theta_{vu}^{P_k}$ indicates the normalized attention weight from node v to neighbor u under P_k, and $\mathcal{N}_v^{P_k}$ represents the neighbor set of v on P_k.

(c) Meta-Path Fusion Functions. They aggregate features across semantic paths to capture the graph's global structure. We define three operations $\mathcal{O}^{(3)} = \{o_1^{(3)}, o_2^{(3)}, o_3^{(3)}\}$.

Mean averages over all meta-paths:

$$\tilde{Z}_v = o_1^{(3)}(\tilde{X}_k^{(2)}) = \frac{1}{K}\sum_{k=1}^{K}\tilde{X}_k^{(2)}, \tag{18}$$

Attention performs weighted aggregation of meta-paths based on their learned importance:

$$\tilde{Z}_v = \sum_{k=1}^{K}\beta_k\tilde{X}_k^{(2)}, \tag{19}$$

$$\beta_k = \frac{\exp(e_k)}{\sum_{j=1}^{K}\exp(e_j)}, \tag{20}$$

$$e_k = \mathbf{w}^T\tanh(\mathbf{W}\tilde{X}_k^{(2)} + \mathbf{b}), \tag{21}$$

where β_k denotes the normalized attention weight, e_k is the attention score, and $\mathbf{w}$, $\mathbf{W}$, $\mathbf{b}$ are learnable parameters.

Multi-Head Attention performs parallel aggregation via R independent attention heads:

$$\tilde{Z}_v = o_3^{(3)}(\tilde{X}_k^{(2)}) = \Big\|_{r=1}^{R}\sigma\left(\sum_{k=1}^{K}\left[\beta_k\right]_r\cdot\tilde{X}_k^{(2)}\right), \tag{22}$$

where $[\beta_k]_r$ is the normalized importance weight of the r-th attention head for the k-th meta-path feature $\tilde{X}_k^{(2)}$, and $\tilde{Z}_v$ is the final aggregated node feature.

3.3.2. Module-level differentiable architecture search

Differentiable Architecture Search (DARTS) performs well on homogeneous GNNs by jointly optimizing all operations within a single continuous search space. However, this global optimization treats the entire architecture as uniform, ignoring the distinct functional roles of different components in heterogeneous graphs.

To address this, we propose a module-level differentiable search framework that decomposes meta-path learning into three semantically aligned modules: message passing, feature aggregation, and meta-path fusion. Each is optimized within its own compact search space. This design enables stage-wise architecture adaptation to heterogeneous semantics, leading to better representation quality, faster convergence, and improved interpretability.

Let W denote network weights (e.g., parameters of MLP/linear layers and attention modules) and $\Xi = \{\xi^{(j,k)}, \gamma\}$ denote architecture parameters. We adopt a softmax-based continuous relaxation to mix candidate operations within each module.

$$\bar{o}^{(1,k)}(\cdot) = \sum_{i=1}^{3}\left[\mathrm{softmax}\left(\xi^{(1,k)}\right)\right]_i o_i^{(1)}(\cdot), \tag{23}$$

$$\bar{o}^{(2,k)}(\cdot) = \sum_{i=1}^{3} \left[\text{softmax}\left(\xi^{(2,k)}\right)\right]_i o_i^{(2)}(\cdot), \tag{24}$$

$$\bar{o}^{(3)}(\cdot) = \sum_{i=1}^{3} \left[\text{softmax}(\gamma)\right]_i o_i^{(3)}(\cdot). \tag{25}$$

Since $\bar{o}^{(j,k)}(\cdot)$ is a softmax-weighted sum, gradients w.r.t. Ξ are obtained through standard backpropagation via the softmax weights.

Given the module-level candidate set in Table 1, we follow the standard DARTS [30] bilevel optimization and optimize W and Ξ by alternating updates. At each iteration $t = 1, \ldots, T$, we sample a training mini-batch $\mathcal{B}_{\text{tr}}$ and a validation mini-batch $\mathcal{B}_{\text{val}}$, and perform the following updates:

$$W \leftarrow W - \eta_W \nabla_W \mathcal{L}_{\text{train}}(W, \Xi; \mathcal{B}_{\text{tr}}), \quad \Xi \leftarrow \Xi - \eta_\Xi \nabla_\Xi \mathcal{L}_{\text{val}}(W, \Xi; \mathcal{B}_{\text{val}}), \tag{26}$$

where we use the first-order DARTS approximation when updating Ξ (i.e., ignoring the unrolled dependency of $W^*(\Xi)$ on Ξ) for efficiency (Fig. 3).

After convergence, we discretize by $o_* = \arg\max_i \text{softmax}(\cdot)_i$ for each module (i.e., for $\xi^{(1,k)}$, $\xi^{(2,k)}$, and γ), and retrain the resulting fixed architecture. The resulting deterministic architecture is:

$$\tilde{Z}_v = o_*^{(3)}\left(o_*^{(2)}\left(o_*^{(1)}\left\{\mathcal{X}_v \mid v \in P_k, \right\}\right), P_k \in \mathcal{S}\right). \tag{27}$$

Compared to the classical DARTS [31] paradigm that treats the entire architecture as a single search space, our module-level design semantically aligns the search process with heterogeneous graph structures, significantly reducing the parameter scale of each optimization step and lowering gradient variance, thereby enabling the architecture search to converge to a stable region within fewer iterations.

4. Experimental results

Datasets and Tasks. In the evaluation of AMP-HGNAS, we used several representative heterogeneous graph datasets for node classification tasks, including DBLP, IMDB, and ACM in the HGB benchmark dataset, as well as the large-scale OGBN-MAG [32] dataset in the OGB Challenge. The dataset statistics are shown in Table 2.

DBLP: This dataset contains 26,128 nodes and 239,566 edges, and the node types include author (A), paper (P), conference (C), etc. The DBLP dataset uses an inductive setting, and all edges are available during training. The target node (author) is divided into three parts: 24% for training, 6% for validation, and 70% for testing. DBLP is ideal for node classification tasks, which aim to predict the author's research field by analyzing the relationships between authors, papers, and conferences.

IMDB: This dataset contains 21,420 nodes and 86,642 edges, and the node types include movie (M), actor (A), director (D), etc. The IMDB dataset also uses an inductive setting, and all edges are available during training. The target node (movie) is divided into three parts: 24% for training, 6% for validation, and 70% for testing. The IMDB dataset is well-suited for developing recommendation algorithms and performing network analysis, such as classifying movies or drawing relationships between actors and movies.

ACM: This dataset contains 10,942 nodes and 547,872 edges, and the node types include paper (P), author (A), conference (C), etc. The ACM dataset uses an inductive setting, and all edges are available during training. The target node (paper) is divided into

Table 1
The search space for adaptive meta-path aggregation.

Function	Operation		
$\mathcal{O}^{(1)}$	Direct Aggregation	Mean Aggregation	MLP Transformation
$\mathcal{O}^{(2)}$	Mean	Max Pooling	Attention
$\mathcal{O}^{(3)}$	Mean	Attention	Multi-Head Attention

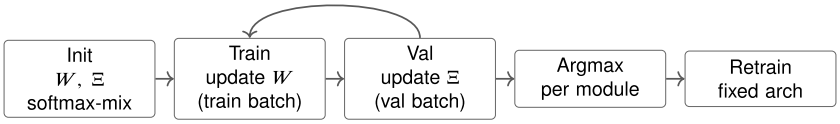


Fig. 3. Module-level differentiable architecture search in Stage II: softmax-mix + alternating updates + argmax.

Table 2
Dataset statistics.

Dataset	#Nodes	#Ntypes	#Edges	#Etypes	Target
DBLP	26 128	4	239 566	6	A
IMDB	21 420	4	86 642	6	M
ACM	10 942	4	547 872	8	P
OGBN-MAG	1 939 743	4	21 111 007	4	P

three parts: 24% for training, 6% for validation, and 70% for testing. The ACM dataset is an important resource for studying academic collaboration networks and knowledge dissemination. Common tasks include predicting the research field of a paper or determining the author's professional field.

OGBN-MAG: This dataset contains 1,939,743 nodes and 21,111,007 edges, and the node types include paper (P), author (A), institution (I), field (F), etc. The OGBN-MAG dataset uses the official division, with papers published before 2018, papers published in 2018, and papers published in 2019 and later as training, validation, and testing nodes, respectively. The main task of this dataset is to predict the publication location of a paper and solve the multi-category classification challenge.

Compared Methods. We compare AMP-HGNAS with a broad range of HGNN baselines, including models based on manually predefined meta-paths, such as NARS, HAN, MAGNN, Simple-HGN, and HetGNN; models based on adaptively searched meta-paths, such as SeHGNN and LMSPS; and models built on other heterogeneous structures, including GTN, GCN, RGCN, DiffMG, and Space4HGNN. Most of these baselines encounter memory overflow issues when handling large-scale datasets such as OGBN-MAG.

It is worth noting that RGCN adopts a new heterogeneous graph interface for reimplementation, significantly reducing memory consumption. NARS uses predefined meta-paths and precomputation to reduce runtime memory usage, avoiding memory exhaustion during large-dataset processing. Additionally, SeHGNN and LMSPS reduce the number of meta-paths through efficient meta-path sampling, thereby lowering memory consumption.

Searching settings. In AMP-HGNAS, the adaptive aggregation search space is divided into three modules: message passing, instance aggregation, and meta-path fusion, each offering three candidate operations. To ensure experimental consistency, we fix the model depth at two layers and follow the bilevel protocol: we update network weights on the training split and update architecture parameters on the validation split (6% for DBLP/IMDB/ACM; official validation for OGBN-MAG). The search strategy follows the Differentiable Module-level Architecture Search Strategy, with operation weights w, architecture parameters α, and their optimizers configured exactly as in previous work [33]. We repeat the search on four datasets using distinct random seeds, training each two-layer GNN for 100 epochs with a batch size of 64.

Training setting. During training, we followed the same setup as in previous works [40], covering data splits, optimizer choice, feature-dimension adjustment, batch normalization, residual connections, and so forth. We used the Adam optimizer with an initial learning rate of 1e-5; whenever the validation loss failed to improve for ten consecutive iterations, the learning rate was halved. To mitigate overfitting, we also set the dropout rate to 0.2 in the adaptive meta-path aggregation module. After selecting the optimal network structure based on the validation set, we repeated the training for multiple runs: each run lasted 500 epochs, and the batch size was fixed at 64. The evaluation metric is the average of Macro-F1 and Micro-F1 on DBLP/IMDB/ACM, while for OGBN-MAG, we followed the official OGB protocol and reported test accuracy. The entire experiment was completed on a single RTX 4080 Super card.

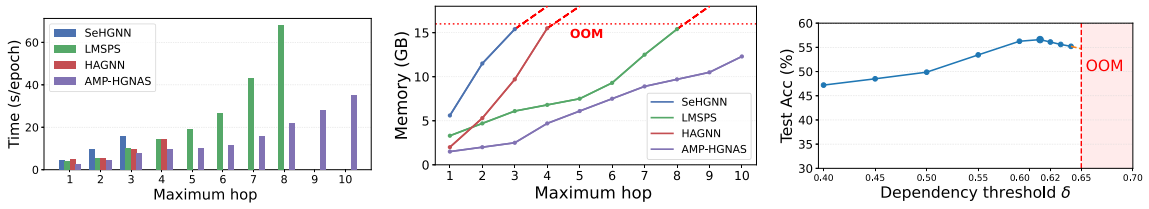
4.1. Results and analysis

Results on node classification task. Table 3 summarizes the node classification results on four heterogeneous graph benchmarks. Overall, AMP-HGNAS achieves the best performance across datasets and shows a clear advantage on the large-scale OGBN-MAG, reaching 56.20% test accuracy compared to 54.83% of LMSPS. In addition, several meta-path-based baselines (e.g., HAN, GTN, HetGNN, MAGNN, and Simple-HGN) encounter out-of-memory (OOM) issues on OGBN-MAG, indicating that naïvely enlarging the receptive field and meta-path space does not scale to large graphs. By explicitly modeling meta-path dependencies, AMP-HGNAS suppresses redundant/overlapping meta-paths and stabilizes aggregation, improving scalability while maintaining strong predictive performance.

Efficiency–performance trade-off on OGBN-MAG. To complement the accuracy comparisons in Table 3, we further report efficiency–performance results on the large-scale OGBN-MAG in Fig. 4. Fig. 4(a) and Fig. 4(b) show that enlarging the receptive field

Table 3
The performance of AMP-HGNAS on both small and large datasets, compared with baselines, is summarized below. The best results are shown in bold, and OOM stands for out of memory.

Method	DBLP		IMDB		ACM		OGBN-MAG
	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Test Acc
RGCN [34]	91.52±0.50	92.07±0.50	58.85±0.26	62.05±0.15	91.55±0.74	91.41±0.75	47.37±0.48
HAN [4]	91.67±0.49	92.05±0.62	57.74±0.96	64.63±0.58	90.89±0.43	90.79±0.43	OOM
GTN [26]	93.52±0.55	93.97±0.54	60.48±0.98	65.14±0.45	91.31±0.70	91.20±0.71	OOM
HetGNN [23]	91.76±0.43	92.33±0.41	48.25±0.67	51.16±0.65	85.91±0.25	86.05±0.25	OOM
MAGNN [5]	93.28±0.51	93.76±0.45	56.49±3.20	64.67±1.67	90.88±0.64	90.77±0.65	OOM
GCN [35]	92.40±0.31	93.03±0.28	58.94±1.35	65.48±0.94	92.26±0.94	92.19±0.43	OOM
Simple-HGN [36]	93.02±0.41	93.80±0.43	60.83±1.46	66.34±0.71	92.54±0.54	92.54±0.49	OOM
DiffMG [37]	94.08±0.17	94.14±0.19	63.55±0.68	67.74±0.38	93.31±0.33	93.14±0.49	OOM
NARS [38]	94.88±0.26	94.92±0.29	63.51±0.51	67.92±0.44	93.89±0.45	93.94±0.43	50.66±0.22
Space4HGNN [39]	94.82±0.26	94.94±0.29	61.57±0.35	67.88±0.31	94.08±0.21	93.77±0.30	OOM
SeHGNN [7]	94.95±0.19	95.06±0.20	64.05±0.60	68.64±0.33	94.06±0.22	94.06±0.19	51.45±0.29
LMSPS [9]	95.35±0.22	95.66±0.20	66.99±0.32	68.70±0.26	94.73±0.41	94.69±0.36	54.83±0.20
CCMG [14]	94.71±0.11	94.34±0.21	64.87±0.68	68.42±0.33	93.12±0.24	93.69±0.23	OOM
HAGNN [8]	95.66±0.31	95.80±0.30	65.57±0.49	68.77±0.23	93.77±0.44	94.29±0.16	53.26±0.37
AMP-HGNAS	**95.83 ± 0.07**	**96.11 ± 0.06**	**67.29 ± 0.30**	**69.51 ± 0.23**	**95.15 ± 0.16**	**95.13 ± 0.16**	**56.20 ± 0.23**



(a) Time cost. (b) GPU memory cost. (c) Threshold δ impact.

Fig. 4. Efficiency–performance trade-offs on OGBN-MAG (large-scale). (a) Training time per epoch under different maximum hop settings. (b) Peak GPU memory usage; increasing hops enlarges the receptive field and amplifies the number of aggregated neighbor/meta-path instances, which may trigger OOM for baselines, whereas AMP-HGNAS remains feasible by keeping the effective meta-path set size M under control via Stage I redundancy pruning. (c) On OGBN-MAG, the best accuracy is achieved at $\delta = 0.61$. When $\delta > 0.61$, accuracy gradually declines, likely because a larger threshold retains more correlated meta-paths, increasing redundancy and computation. When $\delta \geq 0.65$, the enlarged meta-path set may exceed the same GPU memory budget and trigger OOM.

Table 4
Performance comparison by replacing predefined meta-path sets with our searched results. Absolute gains over the corresponding backbone are shown in the second line for each +AMP entry.

	DBLP		IMDB		ACM	
Method	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
SeHGNN	94.95±0.19	95.06±0.20	64.05±0.60	68.64±0.33	94.06±0.22	94.06±0.19
SeHGNN + AMP	95.68±0.25	95.77±0.23	64.10±0.27	68.59±0.24	94.73±0.20	94.76±0.18
	(+0.73)	(+0.71)	(+0.05)	(−0.05)	(+0.67)	(+0.70)
LMSPS	95.35±0.22	95.66±0.20	66.99±0.32	68.70±0.26	94.73±0.41	94.69±0.36
LMSPS + AMP	95.80±0.15	95.97±0.13	67.15±0.20	68.89±0.17	94.95±0.28	94.96±0.26
	(+0.45)	(+0.31)	(+0.16)	(+0.19)	(+0.22)	(+0.27)
CCMG	94.71±0.11	94.34±0.21	64.87±0.68	68.42±0.33	93.12±0.24	93.69±0.23
CCMG + AMP	93.54±0.15	93.21±0.17	65.12±0.48	68.62±0.52	92.15±0.44	92.28±0.47
	(−1.17)	(−1.13)	(+0.25)	(+0.20)	(−0.97)	(−1.41)
Simple-HGN	93.02±0.41	93.80±0.43	60.83±1.46	66.34±0.71	92.54±0.54	92.54±0.49
Simple-HGN + AMP	94.23±0.14	94.14±0.17	60.89±0.32	66.77±0.38	93.11±0.24	93.48±0.23
	(+1.21)	(+0.34)	(+0.06)	(+0.43)	(+0.57)	(+0.94)
HAGNN	95.66±0.31	95.80±0.30	65.57±0.49	68.77±0.23	93.77±0.44	94.29±0.16
HAGNN + AMP	95.79±0.21	95.89±0.26	66.17±0.23	68.89±0.30	94.03±0.19	94.70±0.11
	(+0.13)	(+0.09)	(+0.60)	(+0.12)	(+0.25)	(+0.41)

(more hops) quickly increases time/memory usage and can trigger OOM for several meta-path-based baselines. Mechanistically, the hop controls receptive-field depth while δ controls the retained meta-path set size (M). Stage I prunes redundant meta-paths to keep M bounded, which helps keep Stage II fusion/aggregation feasible under a fixed GPU budget (Fig. 4).

Larger hops induce deeper neighborhood expansion, which inflates the number of traversed instances and messages to be aggregated, thereby increasing intermediate activations and the associated runtime/memory usage. Moreover, overly large δ retains more correlated meta-paths, which bring diminishing returns in accuracy but can exceed the same GPU budget and cause OOM. In contrast, our pruning keeps the effective meta-path set compact and thus maintains both stable performance and feasibility in the non-OOM region, delineating a practical operating range for scalable search/training.

Effectiveness of the search algorithm. To verify that the meta-paths searched by AMP are not overly tied to a specific HGNN design, we conduct a plug-in transferability evaluation: we replace each backbone's predefined meta-path set with AMP-selected meta-paths, while keeping its original aggregation architecture and training protocol unchanged.

Table 4 shows that AMP yields consistent gains on most backbones across DBLP/IMDB/ACM (e.g., SeHGNN, LMSPS, Simple-HGN, and HAGNN), indicating that the searched meta-paths are generally transferable rather than tailored to a particular aggregator. While the absolute improvements can be modest on near-saturated benchmarks (e.g., DBLP/ACM with ∼95% Micro-F1), they are often comparable to or larger than the run-to-run standard deviations, suggesting stable benefits beyond random fluctuations.

We adopt a conservative plug-in protocol in which only the meta-path set is replaced, and the backbone architecture and training pipeline remain unchanged. For methods such as CCMG that tightly couple meta-path construction with co-evolutionary aggregation, this plug-in constraint can limit the achievable improvements, especially on near-saturated benchmarks. Even so, AMP delivers consistent gains across most backbones and datasets, supporting its effectiveness and transferability.

Complexity analysis. Table 5 reports wall-clock time under a unified setting (hop = 7, $\delta = 0.7$ on DBLP/IMDB/ACM, and $\delta = 0.61$ on OGBN-MAG), which is used as the default setting for subsequent tables unless stated otherwise, where Pre is the one-off preprocessing cost (Stage I), and Train is the Stage II training cost, with All = Pre + Train. Here N denotes the number of target nodes, r the number of relation types, d the average per-relation neighbor count, l the number of layers, F the hidden feature dimension, and M the number of retained meta-paths in Stage I. Table 5 shows that AMP-HGNAS achieves the lowest overall time among scalable

Table 5
Wall-clock time breakdown (seconds) and rough asymptotic time complexity under a unified setting. We report preprocessing time (Pre) and training time (Train) separately, and the overall time is All = Pre + Train. For methods without pre-computation, Pre = 0 and thus All = Train. OOM denotes out-of-memory under the same GPU budget.

Method	Time Complexity		DBLP			IMDB			ACM			OGBN-MAG		
	Pre	Train	Pre	Train	All	Pre	Train	All	Pre	Train	All	Pre	Train	All
HAN	–	$O(N(rd)lF^2)$	0	53	53	0	23	23	0	27	27	OOM	OOM	OOM
Simple-HGN	–	$O(N(rd)lF^2)$	0	52	52	0	39	39	0	72	72	OOM	OOM	OOM
SeHGNN	$O(NrlF^2)$	$O(N(rlF^2 + r^2lF))$	23	22	45	18	103	121	9	77	86	OOM	OOM	OOM
LMSPS	$O(NrlF^2)$	$O(NMF^2)$	23	18	41	20	30	50	17	30	47	435	3722	4157
AMP-HGNAS	$O(NMF^2)$	$O(NMF^2)$	13	26	39	12	25	37	11	30	41	206	1625	1831

Table 6
Stage-wise wall-clock time and memory breakdown of AMP-HGNAS under the same setting as Table 5. DG: dependency graph construction (incl. greedy selection). Search: differentiable architecture search with supernet training (including weight-sharing updates and architecture derivation). Agg: post-search fine-tuning of the derived architecture. DG is executed on CPU (CPU Mem: peak resident set size), while Search/Agg runs on GPU (GPU Mem: peak allocated memory).

Dataset	DG		Search		Agg		Total	
	Time(s)	CPU Mem(GB)	Time(s)	GPU Mem(GB)	Time(s)	GPU Mem(GB)	Time(s)	Peak GPU(GB)
DBLP	13	2.8	16	3.2	10	2.8	39	3.2
IMDB	12	2.2	13	3.5	12	3.0	37	3.5
ACM	11	2.7	19	3.8	11	3.2	41	3.8
OGBN-MAG	206	7.5	1105	8.9	520	8.3	1831	8.9

Table 7
Impact of different dependency Thresholds on model performance.

	DBLP		IMDB		ACM	
Thresholds (δ)	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1
0.55	94.60 ± 0.13	94.95 ± 0.12	64.80 ± 0.40	66.20 ± 0.35	94.20 ± 0.22	94.15 ± 0.21
0.60	95.12 ± 0.10	95.45 ± 0.09	66.10 ± 0.35	68.10 ± 0.28	94.80 ± 0.18	94.80 ± 0.18
0.65	95.38 ± 0.09	95.70 ± 0.08	66.60 ± 0.33	68.80 ± 0.26	94.95 ± 0.17	94.95 ± 0.17
0.70 (best)	95.83 ± 0.07	96.11 ± 0.06	67.29 ± 0.30	69.51 ± 0.23	95.15 ± 0.16	95.13 ± 0.16
0.75	95.30 ± 0.08	95.90 ± 0.07	67.00 ± 0.32	69.10 ± 0.25	95.10 ± 0.16	94.70 ± 0.16
0.80	95.20 ± 0.10	95.50 ± 0.09	66.00 ± 0.35	68.00 ± 0.28	94.70 ± 0.18	94.10 ± 0.18

methods: All = 39/37/41s on DBLP/IMDB/ACM, and it remains feasible on OGBN-MAG while SeHGNN runs into OOM under the same GPU budget. On OGBN-MAG, AMP-HGNAS is substantially faster than LMSPS, mainly because Stage I redundancy pruning controls the effective meta-path set size M and reduces repeated message passing over highly overlapping neighborhoods, lowering both computation and activation memory. In our implementation, the differentiable search update in Stage II shares the same forward/backward computation with aggregation by weight sharing, making their wall-clock costs tightly coupled; hence, we report the aggregated Stage II cost as Train and complement it with peak GPU memory in Fig. 4(b). To further complement Table 5, we report a stage-wise wall-clock time and memory breakdown of AMP-HGNAS in Table 6.

Sensitivity analysis. Table 7 studies the effect of δ on DBLP/IMDB/ACM, while Fig. 4(c) highlights the large-scale behavior on OGBN-MAG. We select δ by validation with a coarse grid search; on OGBN-MAG, we additionally restrict δ to a no-OOM feasible range under the GPU budget.

On small/medium benchmarks, the near-optimal δ typically falls within a moderate range and is broadly consistent across datasets: an overly small δ may over-prune and reduce semantic diversity, whereas an overly large δ retains correlated meta-paths and introduces redundant messages. On OGBN-MAG, the preferred δ tends to be slightly smaller, since a larger δ typically increases the retained meta-path count M and can rapidly lead to OOM; thus, we first determine a feasible range and then select the best δ within it based on validation performance.

Training dynamics. Fig. 5 plots the validation Macro-F1 after each training epoch to illustrate the optimization dynamics. On DBLP/IMDB/ACM, AMP-HGNAS improves more rapidly in the early epochs and converges to a higher plateau, while the baselines converge more slowly and show larger mid-training fluctuations (especially on ACM). On the large-scale OGBN-MAG, all methods progress more gradually and saturate around 0.5–0.56; nevertheless, AMP-HGNAS remains consistently competitive and achieves the best final region. We report Macro-F1 to weight all classes equally and reduce bias toward frequent ones on imbalanced heterogeneous graphs. Overall, the figure indicates that AMP-HGNAS improves not only the final performance but also convergence speed and training stability.

Ablation study. Fig. 6 evaluates the necessity of the two stages by removing Stage I (dependency-aware meta-path search) or Stage II (adaptive aggregation via module-level NAS). Performance drops in both cases: removing Stage I retains highly dependent meta-paths, introducing redundant messages, and reducing effective semantic diversity, which leads to a pronounced performance

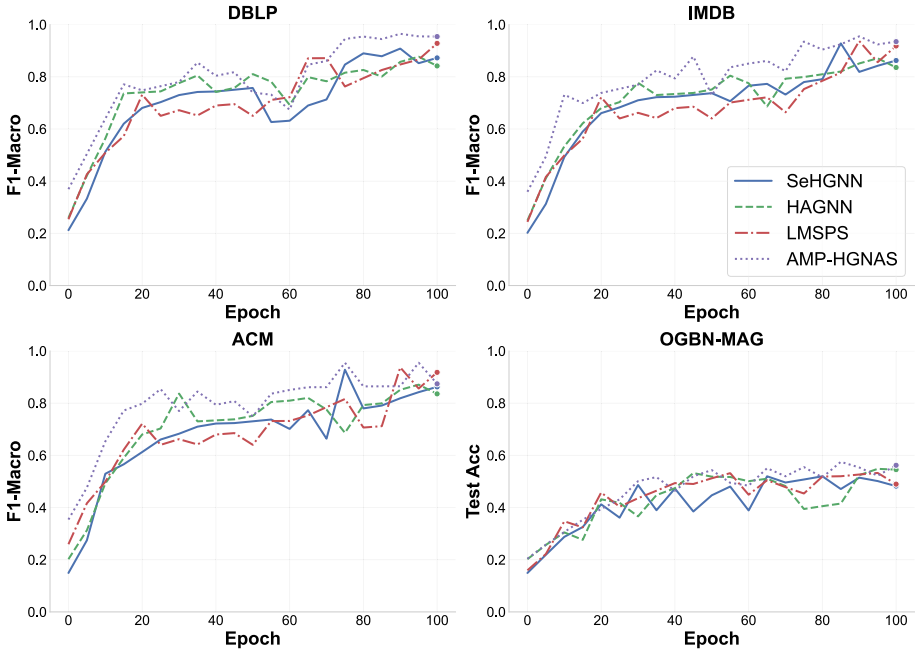


Fig. 5. Validation trajectories over 100 epochs on DBLP, IMDB, ACM, and OGBN-MAG. One epoch denotes one full pass over the training set; we report validation Macro-F1 for DBLP/IMDB/ACM and validation accuracy for OGBN-MAG after each epoch.

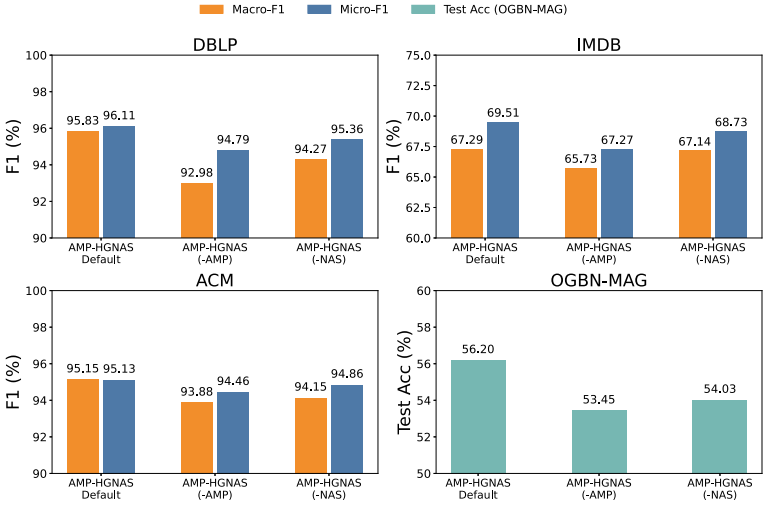


Fig. 6. Ablation study of Stage I and Stage II on four datasets. For DBLP/IMDB/ACM, we report both Macro-F1 and Micro-F1. For OGBN-MAG, we follow the OGB benchmark protocol and report test accuracy instead of F1. Without Stage I replaces dependency-aware meta-path search with a commonly used manual meta-path set, and Without Stage II replaces module-level differentiable NAS with a fixed aggregation module (regular attention).

drop; removing Stage II replaces adaptive NAS-based fusion with a fixed aggregator, which cannot adapt the fusion choice to dataset-specific semantics, resulting in suboptimal aggregation and degraded performance.

To provide a finer-grained ablation for Stage I, Table 8 summarizes the meta-path selection results under a unified setting, where we replace the Jaccard similarities in Eq. (1) with cosine similarity or the overlap coefficient while keeping the rest unchanged. Under the same hop and δ, Greedy + Jaccard achieves the best results on all datasets. Replacing the default Jaccard overlap with cosine similarity or the overlap coefficient leads to a clear drop (e.g., OGBN-MAG 56.20 → 54.88/54.81), and alternative selection rules are inferior on large graphs (Random + Jaccard 53.78, Degree + Jaccard 54.05), showing that both the dependency metric and the greedy non-adjacent selection are important for retaining a diverse yet non-redundant meta-path set.

To further provide a finer-grained ablation within Stage II, Table 9 freezes one Stage II module at a time while keeping the other two searchable. Freezing $O^{(1)}$ (message passing) yields the largest drop, especially on OGBN-MAG (Test Acc 56.20 → 54.37), indicating that learnable propagation is the main source of Stage II gains. Freezing $O^{(2)}$ (instance aggregation) causes a moderate but consistent degradation (e.g., OGBN-MAG 56.20 → 55.33), showing the benefit of adaptive instance-level aggregation. Freezing $O^{(3)}$

Table 8
Component-level ablation within Stage I. We fix Stage II and compare different sampling strategies and dependency metrics for constructing the dependency graph under the unified setting.

Sampling	Metric	DBLP		IMDB		ACM		OGBN-MAG
		Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Test Acc
Greedy	Jaccard	**95.83 ± 0.07**	**96.11 ± 0.06**	**67.29 ± 0.30**	**69.51 ± 0.23**	**95.15 ± 0.16**	**95.13 ± 0.16**	**56.20 ± 0.23**
	Cosine	93.58 ± 0.17	95.85 ± 0.21	66.40 ± 0.22	66.08 ± 0.20	93.68 ± 0.12	93.92 ± 0.22	54.88 ± 0.16
	Overlap	93.66 ± 0.23	95.87 ± 0.17	66.32± 0.11	67.10 ± 0.33	93.65 ± 0.25	93.74 ± 0.20	54.81 ± 0.17
Random	Jaccard	95.55 ± 0.27	94.69 ± 0.21	66.38 ± 0.09	65.75 ± 0.17	92.25 ± 0.10	92.89 ± 0.12	53.78 ± 0.16
	Cosine	94.83 ± 0.22	93.88 ± 0.12	61.35 ± 0.10	64.42 ± 0.20	93.03 ± 0.09	92.37 ± 0.14	52.36 ± 0.13
	Overlap	94.54 ± 0.21	94.81 ± 0.17	63.37 ± 0.15	65.04 ± 0.11	92.64 ± 0.18	93.58 ± 0.28	52.89 ± 0.29
Degree (Min-deg)	Jaccard	94.52 ± 0.33	94.99 ± 0.27	63.73 ± 0.12	66.00 ± 0.33	92.65 ± 0.23	93.35 ± 0.11	54.05 ± 0.07
	Cosine	93.66 ± 0.20	93.70 ± 0.21	63.43 ± 0.17	65.77 ± 0.18	93.20± 0.16	92.33 ± 0.09	53.73 ± 0.22
	Overlap	93.91 ± 0.24	93.78 ± 0.13	64.22 ± 0.11	65.34 ± 0.14	92.45 ± 0.25	92.84 ± 0.20	53.34 ± 0.19

Table 9
Component-level ablation within Stage II. We fix Stage I and ablate the three Stage II modules by freezing one module at a time. ✓ means the module is kept but frozen, while × means it is searched by NAS. Here $O^{(1)}$, $O^{(2)}$, and $O^{(3)}$ correspond to the three operation sets in Table 1. When a module is frozen, we replace its mixed operation with the same fixed operator used in the Without Stage II setting for a fair comparison.

ID	Frozen Module?			DBLP		IMDB		ACM		OGBN-MAG
	Freeze $\mathcal{O}^{(1)}$	Freeze $\mathcal{O}^{(2)}$	Freeze $\mathcal{O}^{(3)}$	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Macro-F1	Micro-F1	Test Acc
1	×	×	×	95.83 ± 0.07	96.11 ± 0.06	67.29 ± 0.30	69.51 ± 0.23	95.15 ± 0.16	95.13 ± 0.16	56.20 ± 0.23
2	✓	×	×	94.73 ± 0.22	95.44 ± 0.19	66.57 ± 0.27	68.75 ± 0.25	94.27 ± 0.09	94.79 ± 0.21	54.37 ± 0.21
3	×	✓	×	94.87 ± 0.20	95.51 ± 0.11	67.15 ± 0.23	68.87 ± 0.15	94.41 ± 0.10	94.90 ± 0.07	55.33 ± 0.17
4	×	×	✓	95.37 ± 0.15	95.74 ± 0.11	67.22 ± 0.15	69.03 ± 0.20	94.55 ± 0.09	95.03 ± 0.12	56.02 ± 0.13

(meta-path fusion) has the smallest impact and remains close to full search, suggesting fusion mainly provides a mild refinement once per-meta-path representations are formed.

5. Conclusion

In this work, we propose an innovative heterogeneous graph neural architecture search model, AMP-HGNAS. This model builds a more general and flexible framework, aiming to adaptively explore effective meta-path combinations and design personalized aggregation strategies based on different datasets and downstream task requirements, and can effectively avoid interdependencies between selected meta-paths. In addition, it can adaptively select the best meta-path aggregation method for different downstream tasks with the help of architecture search technology. Experimental results show that AMP-HGNAS demonstrates its superiority in a variety of heterogeneous graph learning tasks, showcasing its capability to dynamically adjust meta-path aggregation strategies to adapt to different tasks.

Several aspects can be further improved in future work. First, our dependency score mainly captures structural redundancy via node and edge overlap, and incorporating representation-level or task-aware similarity could better reflect semantic redundancy between meta-paths. Second, although we provide an empirical sensitivity analysis for the redundancy threshold, the optimal threshold can still be dataset dependent, especially when graph scale and semantic structure vary, and an adaptive thresholding strategy is an interesting direction for future work. Third, although the two-stage design and weight sharing make the search feasible on large graphs in our experiments, the overall search cost still scales with the candidate meta-path space and hop budget, and further acceleration could improve scalability under stricter memory and time budgets. Finally, our evaluation is conducted on several widely used heterogeneous graph benchmarks, and for graphs with substantially different semantic structures, the notion of redundancy and the optimal meta-path patterns may change, so it remains valuable to benchmark AMP-HGNAS on such domains and to develop schema- and relation-aware dependency modeling to enhance cross-domain generalization.

CRediT authorship contribution statement

Xinsheng Li: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Pietro Lió:** Writing – review & editing, Supervision, Project administration, Conceptualization. **Lintao Yang:** Supervision, Software, Resources, Conceptualization. **Zhigang Ye:** Validation, Software, Resources, Conceptualization. **Chengbin Peng:** Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. List of acronyms and glossary

Term / Acronym	Plain explanation
HGNN	A GNN for graphs with multiple node/edge types. Example: in DBLP, nodes can be Author/Paper/Venue, and edges mean "write" or "publish".
Meta-path	A template of typed relations that defines a specific way two nodes can be connected. Example: A–P–A means two authors are connected if they wrote the same paper.
Meta-path dependency	Two meta-paths are "too similar" if they reach almost the same neighbors/edges, so using both may repeat the same information. Example: if A–P–A and A–P–V–P–A bring nearly the same author neighborhood, one may be redundant.
Dependency threshold δ	A cutoff that decides when two meta-paths are considered redundant enough to be linked in the dependency graph. Example: if overlap score $D(P_i, P_j) > \delta$, we consider the two meta-paths to be highly dependent.
Meta-path fusion strategy	A scheme to fuse the representations produced by different meta-path branches into a single node embedding. It decides how much each meta-path contributes.
Differentiable module-level NAS	A gradient-based architecture search that assigns each candidate operation a continuous gate, So the operation choices can be optimized by backpropagation.
Hop	How far do we traverse neighbors when collecting information. Example: 1-hop uses direct neighbors; 2-hop also includes neighbors-of-neighbors.
Receptive field	The set of nodes that can affect a target node's representation. Example: increasing hop/layers expands the receptive field but also increases time/memory.
Cosine similarity	Measures similarity by the angle between two coverage vectors, focusing on direction rather than exact overlap.
Overlap coefficient	Measures overlap normalized by the smaller set: $\frac{\lvert A \cap B \rvert}{\min(\lvert A \rvert, \lvert B \rvert)}$ (applied to node/edge sets).

Data availability

Data will be made available on request.

References

[1] Y. Sun, J. Han, X. Yan, et al., Mining heterogeneous information networks: a structural analysis approach, in: Proceedings of the 12th International Conference on Data Mining (ICDM 2012), 2012, pp. 48–57.
[2] R. Bing, G. Yuan, M. Zhu, Heterogeneous graph neural networks analysis: a survey of techniques, evaluations and applications, Artif. Intell. Rev. 56 (8) (2023).
[3] C. Shi, Y. Li, J. Zhang, A survey of heterogeneous information network analysis, IEEE Trans. Knowl. Data Eng. 29 (1) (2016) 17–37.
[4] X. Wang, H. Ji, C. Shi, et al., Heterogeneous graph attention network, in: Proceedings of the ACM Web Conference, WWW, 2019, pp. 2022–2032.
[5] X. Fu, J. Zhang, Z. Meng, et al., Magnn: metapath aggregated graph neural network for heterogeneous graph embedding, in: Proceedings of the Web Conference 2020, 2020, pp. 2331–2341.
[6] X. Fu, I. King, Mecch: metapath context convolution-based heterogeneous graph neural networks, Neural Netw. 170 (2024) 266–275.
[7] X. Yang, M. Yan, S. Pan, et al., Simple and efficient heterogeneous graph neural network, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, 2023.
[8] G. Zhu, Z. Zhu, H. Chen, et al., Hagnn: hybrid aggregation for heterogeneous graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 36 (8) (2025) 14536–14550.
[9] C. Li, Z. Guo, Q. He, Long-range meta-path search on large-scale heterogeneous graphs, in: Proceedings of the Thirty-Eighth Annual Conference on Neural Information Processing Systems, 2020.
[10] Y. Gao, P. Zhang, Z. Li, Heterogeneous graph neural architecture search, in: 2021 IEEE International Conference on Data Mining (ICDM), 2021, pp. 1066–1071.
[11] Y. Gao, P. Zhang, C. Zhou, Hgnas++: efficient architecture search for heterogeneous graph neural networks, IEEE Trans. Knowl. Data Eng. 35 (9) (2023) 9448–9461.
[12] R. Girshick, J. Donahue, T. Darrell, et al., Rich feature hierarchies for accurate object detection and semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2014, pp. 580–587.
[13] Z. Han, F. Xu, J. Shi, Genetic meta-structure search for recommendation on heterogeneous information network, in: Proceedings of the 29th ACM International Conference on Information & Knowledge Management, 2020, pp. 455–464.
[14] Y. Liu, X. Teng, J. Liu, Cooperative co-evolutionary search for meta multigraph and graph neural architecture on heterogeneous information networks, Appl. Soft Comput. 182 (2025) 113541.
[15] S. Liu, C. Fan, K. Cheng, et al., Inductive meta-path learning for schema-complex heterogeneous information networks, IEEE Trans. Pattern Anal. Mach. Intell. 46 (12) (2024) 10196–10209.
[16] M. Nunes, G.L. Pappa, Neural architecture search in graph neural networks, in: Intelligent Systems: 9th Brazilian Conference, BRACIS 2020, Rio Grande, Brazil, October 20–23, 2020, Proceedings, Part I, Springer International Publishing, 2020, pp. 302–317.
[17] P. Ren, Y. Xiao, X. Chang, et al., A comprehensive survey of neural architecture search: challenges and solutions, ACM Computing Surveys (CSUR) 54 (4) (2021) 1–34.
[18] Y. Gao, H. Yang, P. Zhang, Graph neural architecture search, in: Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI 2021), 2021.
[19] L. Yang, P. Lió, X. Shen, et al., Adaptive multi-scale graph neural architecture search framework, Neurocomputing 599 (2024) 128094.
[20] W. Ning, R. Cheng, J. Shen, et al., Automatic meta-path discovery for effective graph-based recommendation, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management, 2022, pp. 1563–1572.

[21] M. Guan, X. Cai, J. Shang, Hmsg: heterogeneous graph neural network based on metapath subgraph learning, Knowl.-based Syst. 279 (2023) 110930.
[22] Y. Chang, C. Chen, W. Hu, Megnn: meta-path extracted graph neural network for heterogeneous graph representation learning, Knowl.-based Syst. 235 (2022) 107611.
[23] C. Zhang, D. Song, C. Huang, et al., Heterogeneous graph neural network, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 793–803.
[24] M.S. Ramkumar, M. Sivaramkrishnan, G.C.P. Latha, Semantic-and relation-aware heterogeneous graph neural network for secure routing in vanets to avoid blackhole attacks, in: 2025 International Conference on Machine Learning and Autonomous Systems (ICMLAS), IEEE, 2025, pp. 1278–1284.
[25] Z. Hu, Y. Dong, K. Wang, et al., Heterogeneous graph transformer, in: Proceedings of the Web Conference 2020, 2020, pp. 2704–2710.
[26] S. Yun, M. Jeong, R. Kim, et al., Graph transformer networks, in: Advances in Neural Information Processing Systems, NeurIPS, 2019, pp. 11960–11970.
[27] Q. Liu, C. Long, J. Zhang, et al., Aspect-aware graph attention network for heterogeneous information networks, IEEE Trans. Neural Netw. Learn. Syst. 35 (5) (2022) 7259–7266.
[28] Z. Han, X. Jin, H. Xing, An effective heterogeneous information network representation learning framework, Future Gener. Comput. Syst. 148 (2023) 66–78.
[29] G. Mei, L. Pan, S. Liu, Heterogeneous graph embedding by aggregating meta-path and meta-structure through attention mechanism, Neurocomputing 468 (2022) 276–285.
[30] H. Zhang, K. Hao, L. Gao, et al., Enhanced gradient for differentiable architecture search, IEEE Trans. Neural Netw. Learn. Syst. 35 (7) (2023) 9606–9620.
[31] C. Li, H. Xu, K. He, Differentiable meta multigraph search with partial message propagation on heterogeneous information networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37(7), 2023, pp. 8518–8526.
[32] W. Hu, M. Fey, M. Zitnik, et al., Open graph benchmark: datasets for machine learning on graphs, Advances in Neural Information Processing Systems, NeurIPS 33 (2020) 22118–22133.
[33] H. Liu, K. Simonyan, Y. Yang, Darts: Differentiable architecture search, arXiv preprint arXiv:1806.09055, 2018.
[34] M. Schlichtkrull, T.N. Kipf, P. Bloem, Modeling relational data with graph convolutional networks, in: The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings, vol. 15, Springer International Publishing, 2018, pp. 593–607.
[35] S. Zhang, H. Tong, J. Xu, et al., Graph convolutional networks: a comprehensive review, Comput. Soc. Netw. 6 (1) (2019) 1–23.
[36] Q. Lv, M. Ding, Q. Liu, et al., Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks, in: KDD '21: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2021, pp. 1150–1160.
[37] Y. Ding, Q. Yao, H. Zhao, et al., Diffmg: differentiable meta graph search for heterogeneous graph neural networks, in: KDD '21: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2021, pp. 279–288.
[38] L. Yu, J. Shen, J. Li, et al., Scalable graph neural networks for heterogeneous graphs, arXiv preprint arXiv:2011.09679, 2020.
[39] T. Zhao, C. Yang, Y. Li, et al., Space4hgnn: A scalable and efficient heterogeneous graph neural network for large-scale heterogeneous graphs, arXiv preprint arXiv:2206:14612, 2022.
[40] Y. Zhang, X. Shen, Y. Xie, Directed link prediction using GNN with local and global feature fusion, IEEE Trans. Netw. Sci. Eng. 12 (1) (2024) 409–422.